Liste

Table of contents

Liste **1**
Liste . . . 1
Creazione di una lista . . . 2
La funzione `range` . . . 2
Indicizzare una lista . . . 3
Aggiungere ed eliminare elementi . . . 4
Le liste come collezioni MUTABILI: cosa succede in memoria? . . . 6
Il concetto di Alias . . . 7
Uguaglianza fra liste . . . 9
Recuperare più elementi da una lista: lo slicing . . . 10
Creare copie di una lista . . . 11
Shallow copy . . . 11
Deep Copy . . . 15
Principali metodi e funzioni per le liste . . . 16
Considerazioni finali . . . 17
Considerazioni finali . . . 18

Liste

Liste

Le liste sono **sequenze ordinate e modificabili** che raggruppano vari oggetti insieme. Per creare una lista, si usa un'assegnazione con una sequenza di oggetti separati da virgole tra parentesi quadre (`[]`):

```
# Define an empty list
empty = []


# Define a list of numbers
```

```
numbers = [1, 2, 3, 100]



# Define a list of strings
superheroes = ["batman", "superman", "spiderman"]


# Define a list of objects with different data types
mixed_types = ["Hello World", [4, 5, 6], False]
```

Creazione di una lista

Alcuni modi per creare una lista

- Lista vuota:

```
a = []
# oppure
b = list()
```

- Lista con elementi:

```
a = [1, 2, 3]
# oppure
b = ['a', 'ciao', 3]
```

- Lista da un'altra stuttura dati:

```
lista = list("ciao")     # ['c', 'i', 'a', 'o']
```

- Mediante la funzione `range`

La funzione `range`

La funzione `range()` restituisce un oggetto iterabile che produce una sequenza di numeri interi. La sintassi è `range(start, stop, step)`

- `start`: valore iniziale della sequenza (incluso). Default: 0
- `stop`: valore finale (escluso)
- `step`: incremento tra un elemento e il successivo. Default: 1

L'oggetto `range` non è una lista, ma immagazzina i parametri essenziali per la creazione di una lista di interi (il primo elemento, lo step, l'ultimo elemento). L'oggetto `range` può essere convertito in una lista tramite il comando `list()`[1].

```
a1 = list( range(10))
# start==0, stop==10 (escluso), step==1
#[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

a2 = list( range (3, 10))
# start==3, stop==10 (escluso), step==1
#[3, 4, 5, 6, 7, 8, 9]

a3 = list( range (3, 10,2))
# start==0, stop==10 (escluso), step==2
#[3, 5, 7, 9]
```

https://docs.python.org/2/library/functions.html#range

Indicizzare una lista

Una **lista** è un contenitore (una *collection*) che consente di gestire un insieme di valori, sia come blocco unico sia accedendo ai singoli elementi uno alla volta. Gli elementi di una lista non devono necessariamente essere omogenei: possiamo combinare numeri interi, numeri decimali (float), stringhe e persino altre liste.

Possiamo accedere al singolo elemento di una lista identificandolo mediante **l'indice di posizione**. L'indice di posizione vale 0 per il primo elemento e vale `len(lista) - 1` per l'ultimo elemento.

Mediante l'indicizzazione possiamo accedere ai singoli elementi sia per leggerne il valore, sia per modificarlo.

Se cerchiamo di accedere ad un indice inesistente otteniamo un errore:

`IndexError: list index out of range`

```
# Indicizzare
a = [1,2,3]
i=0

print(a[i]) # primo elemento
```

[1] tecnicamente `list` è una classe built-in, come vedremo in seguito

```
print(a[len(a)-1]) # ultimo elemento


i=1
# modifico il secondo elemento 'in place',
# cioè sulla lista stessa, senza crearne una nuova
a[i] = 10
print(a[i]) # stampo il secondo elemento

#Se cerchiamo di accedere ad un indice inesistente otteniamo un errore:
print(a[5])
#`IndexError: list index out of range`

# NB anche print(a[3]) produce lo stesso errore: perché?
```

Possiamo anche utilizzare una sintassi semplificata che permette di riferirsi agli elementi della lista a partire dall'ultimo. L'ultimo elemento avrà indice `-1`, il penultimo `-2` e così via.

Aggiungere ed eliminare elementi

Le 'operazioni' richiamate su un oggetto mediante l'operatore punto (`.`) si chiamano METODI - approfondiremo meglio questo concetto più avanti.

Il metodo `append()` consente di aggiungere un singolo elemento alla fine della lista.

Il metodo `extend()` permette di aggiungere alla lista tutti gli elementi di un'altra lista, uno dopo l'altro.

```
a = [1, 2, 3]

b = [4, 5, 6]


a.append(10)

a.extend(b)

print(a) #  [1, 2, 3, 10, 4, 5, 6]
```

L'operazione `a.extend(b)` **aggiunge** tutti gli elementi di `b` ad `a`. Possiamo ottenere lo stesso risultato aggiungendo gli elementi uno per uno, ad esempio con un ciclo:

```
while i<len(b):
  el = b[i]
  a.append(el)
```

Questa modalità ci consente di avere maggiore controllo sull'aggiunta degli elementi. Ad esempio, possiamo scegliere di aggiungere solo i valori pari.

L'operazione

```
 c = a + b
```

concatena gli elementi delle liste `a` e `b`, creando una **nuova lista** `c`. Le liste originali `a` e `b` non vengono modificate.

Per eliminare un elemento da una lista, dato l'indice, posso usare `del`, `remove` o `pop`

```
a = [1, 2, 3, 4, 5, 3, 10]
a.remove(3)  # Rimuove la PRIMA occorrenza di 3
# se il valore cercato non esiste, segnala un errore


a = [1, 2, 3, 4, 5, 3, 10]
a.pop(3)  # Rimuove l'elemento all'indice 3 (il 4 in questo caso)
a.pop()  # Rimuove l'ultimo elemento della lista (il 10 in questo caso)


a = [1, 2, 3, 4, 5, 3, 10]

del a[3] # Rimuove l'elemento all'indice 3 (il 4 in questo caso)
```

Sia `del` sia `pop()` permettono di eliminare elementi da una lista, ma con una differenza sostanziale.

- `del a[i]` rimuove l'elemento di indice `i` e non restituisce un valore
- `a.pop(i)` rimuove l'elemento di indice `i` e lo restituisce:

```
returned_value = a.pop(i)
print(returned_value)
```

- `remove` invece non restituisce nessun valore. Si noti che `remove` accetta in ingresso un valore da cercare nella lista, mentre `del()` e `pop()` accettano un indice.

Come detto, gli elementi di una lista non devono necessariamente essere omogenei: possiamo combinare numeri interi, numeri decimali (float), stringhe e persino altre liste.

`a= [1,  [10, 20, 30], 3]`

In questo caso, 'a[1] è il secondo elemento della lista

`a[1] == [10, 20, 30]`

Posso indicizzare l'elemento interno come indicizzo una normale lista

```
a= [1,  [10, 20, 30], 3]

print(a[1][0] ) # 20

b= a[1][2]
print(a[1][2] ) # 30
```

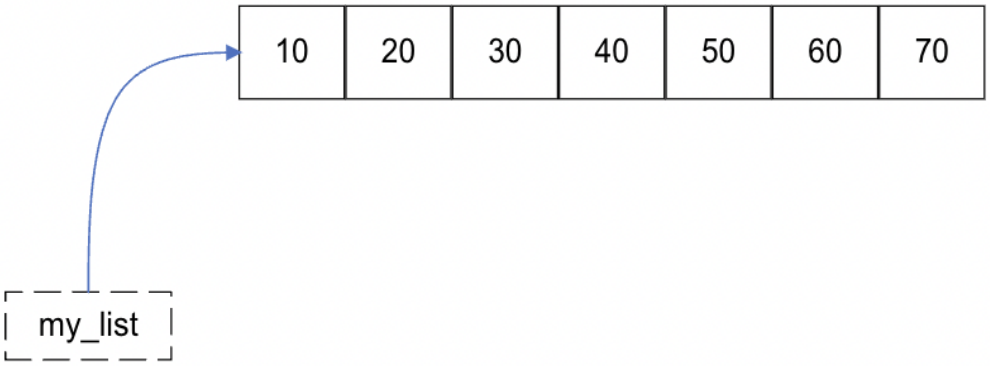
In questo caso `a[1]` indca la lista `[10, 20, 30]` e `a[1][2]` indica l'elemento di indice 2 della lista `[10, 20, 30]`

Le liste come collezioni MUTABILI: cosa succede in memoria?

`my_list = [10, 20, 30, 40, 50, 60, 70]`

Quando creiamo una lista, gli elementi vengono memorizzati in un'area di memoria e alla variabile viene assegnato un riferimento a questa posizione di memoria, proprio come avviene per le altre variabili di tipo semplice.

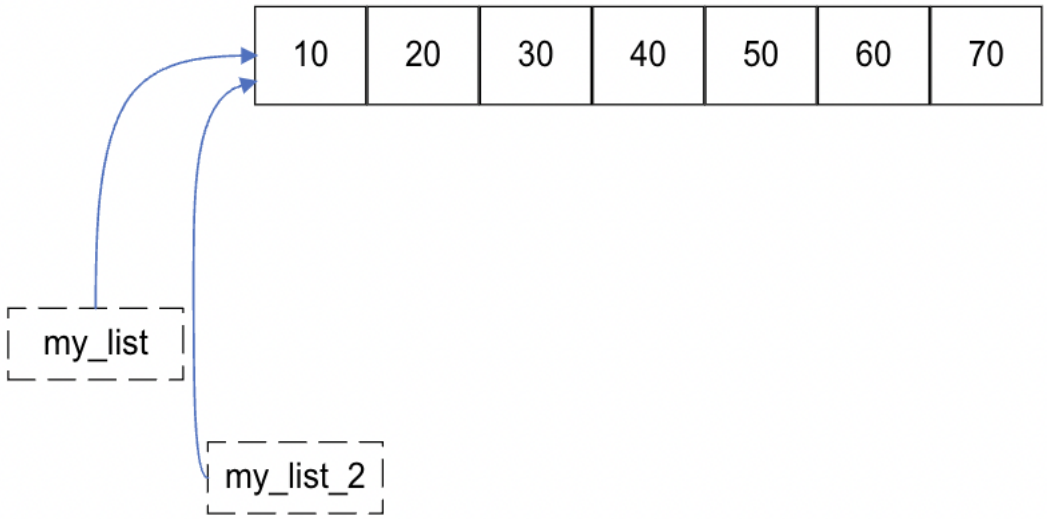
```
my_list = [10, 20, 30, 40, 50, 60, 70]
```



Il concetto di Alias

L'operazione `my_list_2 = my_list` assegna a `my_list_2` lo stesso riferimento in memoria di `my_list`. Non viene creata una nuova lista, ma semplicemente un altro nome che punta alla stessa lista in memoria. Viene stabilito un collegamento (*binding*) tra la variabile `my_list_2` e l'oggetto originale.

Questo spiega perché l'istruzione `my_list[0] = 100` *sembra* modificare entrambe le liste. In realtà, c'è una sola lista. Poiché entrambe le variabili fanno riferimento alla stessa area di memoria, ogni modifica a un elemento della lista è visibile da entrambe le variabili.



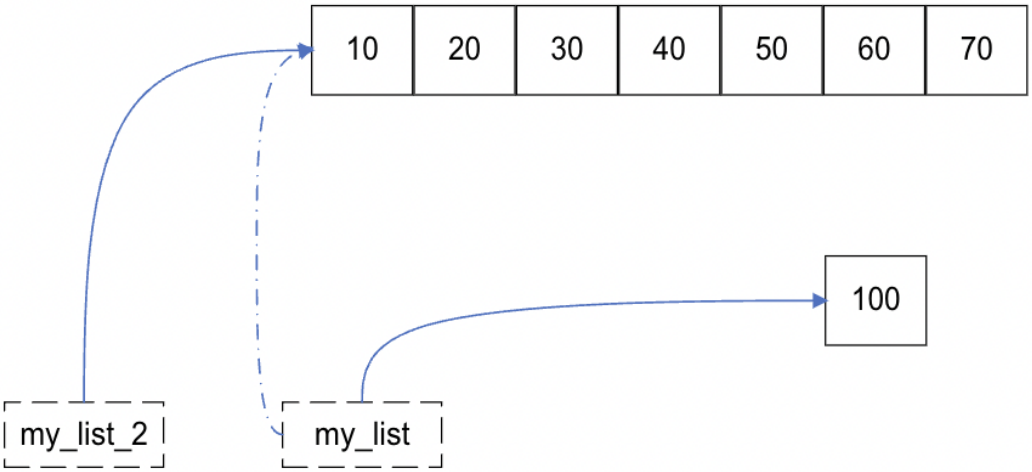
E' semplice verificare che i due identificatori `my_list` e `my_list_2` si riferiscono allo stesso oggetto in memoria:

```
my_list = [10, 20, 30, 40, 50, 60, 70]
my_list_2 = my_list
print(id(my_list))
print(id(my_list_2))
```

Consideriamo le seguenti istruzioni

```
my_list = [10, 20, 30, 40, 50, 60, 70]
my_list_2 = my_list
my_list = 100
print(my_list)
print(my_list_2)
print(id(my_list))
print(id(my_list_2))
```

L'ultima istruzione fa sì che `my_list` non punti più alla lista originale, ma ora faccia riferimento al valore `100`.



Uguaglianza fra liste

Due liste sono considerate uguali se contengono gli *stessi elementi* nello *stesso ordine*

```
l1 = [1,2,3]
l2 = [1,2,3] # stessa sequenza di valori di l1
l3 = [3, 1, 2] # stessi valori, ordine diverso

print(l1==l2) #True
print(l1==l3) #False
```

tuttavia, non per questo sono lo **stesso oggetto** in memoria.

Per verificare invece se due variabili puntano allo stesso oggetto (stessa locazione in memoria), si usa l'operatore `is` (o si confrontano gli `id`):

```
print(l1 is l2)            # False: non è lo stesso oggetto
print(hex(id(l1)), hex(id(l2))     # due valori diversi, perché sono due oggetti distinti
```

Recuperare più elementi da una lista: lo slicing

Un'esigenza comune quando si lavora con le liste è quella di estrarre una porzione, o sottolista, da una lista esistente. Questo è possibile grazie a un'operazione chiamata slicing, che utilizza la seguente sintassi:

`lista[start:stop:step]`

La parte `[start:stop:step]` è nota come operatore di slicing. La sua sintassi prevede una coppia di parentesi quadre e fino a tre indici opzionali: `start`, `stop` e `step`,

- `start`: indica l'indice da cui iniziare lo slicing. L'elemento a questo indice **è incluso** nel risultato.
- `stop`: indica l'indice in cui terminare lo slicing. L'elemento a questo indice **non è incluso** nel risultato.
- `step`: rappresenta il numero di elementi da saltare ad ogni passo

L'istruzione `lista[2:10]` estrae una sottolista che include gli elementi a partire dall'indice 2 fino all'indice 10, **escludendo** l'elemento all'indice 10

L'istruzione `lista[2:10:3]` estrae una sottolista a partire dall'indice 2 fino all'indice 10 (escluso), procedendo con uno step di 3. In questo caso verranno selezionati gli elementi agli indici 2, 5 e 8, ovvero `lista[2]`, `lista[5]` e `lista[8]`.

Tutti e tre gli indici sono *facoltativi* e, se omessi, assumono i seguenti valori predefiniti:

- `start`: 0 (inizio della sequenza)
- `stop`: lunghezza della sequenza
- `step`: 1 (nessun salto, elementi consecutivi)

Esempio (si riportano i casi più comuni)

```
#
lista2 = lista[2:10:3] # dall'indice 2 all'indice 10 (escluso), con step di 3
lista2 = lista[2:10] # dall'indice 2 all'indice 10 (escluso), con step di 1
lista2 = lista[2:] # dall'indice 2 al termine della lista, con step di 1
lista2 = lista[:10] # dall'inizio della lista all'indice 10 (escluso)

lista2 = lista[:]
# tutti gli elementi della lista
# (omettendo `start`, `stop` e `step`, questi assumono i valori di defaul)

lista2 = lista[10:2:-1] # dall'indice 10 all'indice 2 (escluso), con step di -1
lista2 = lista[::-1]  # Tutti gli elementi in ordine inverso
```

Tabella riassuntiva per lo slicing:

Sintassi	Descrizione	Esempio / Note
`lista[start:stop]`	Estrae elementi da `start` a `stop` (escluso)	`lista[2:5]` → elementi in pos. 2,3,4
`lista[start:]`	Estrae dal punto `start` fino alla fine	`lista[2:]` → dal 2° elemento in poi
`lista[:stop]`	Estrae dall'inizio fino a `stop` (escluso)	`lista[:4]` → primi 4 elementi
`lista[:]`	Copia completa della lista	Crea una **copia superficiale**
`lista[start:stop:step]`	Estrae da `start` a `stop` (escluso), saltando di `step` elementi	`lista[1:7:2]` → pos. 1, 3, 5
`lista[::step]`	Estrae tutta la lista con passo `step`	`lista[::2]` → un elemento ogni due
`lista[::-1]`	Estrae la lista al contrario	Inversione rapida
`lista[start::-1]`	Estrae da `start` all'inizio, all'indietro	`lista[4::-1]` → da pos. 4 a 0
`lista[:stop:-1]`	Estrae dalla fine fino a `stop` (escluso), all'indietro	`lista[:2:-1]` → da fine a pos. 3
`lista[start:stop:-1]`	Estrae da `start` a `stop` (escluso), all'indietro	`lista[5:1:-1]` → pos. 5, 4, 3, 2

Creare copie di una lista

Come abbiamo già visto, in Python ad ogni oggetto è associato un **identificatore univoco**, ottenibile tramite la funzione `id()`. Nella maggior parte delle implementazioni di Python (come CPython), la funzione `id()` rende l'indirizzo di memoria in cui l'oggetto è memorizzato.

In Python esistono due modalità principali per creare copie di una lista:

- copia superficiale (**shallow copy**)
- copia profonda (**deep copy**)

Shallow copy

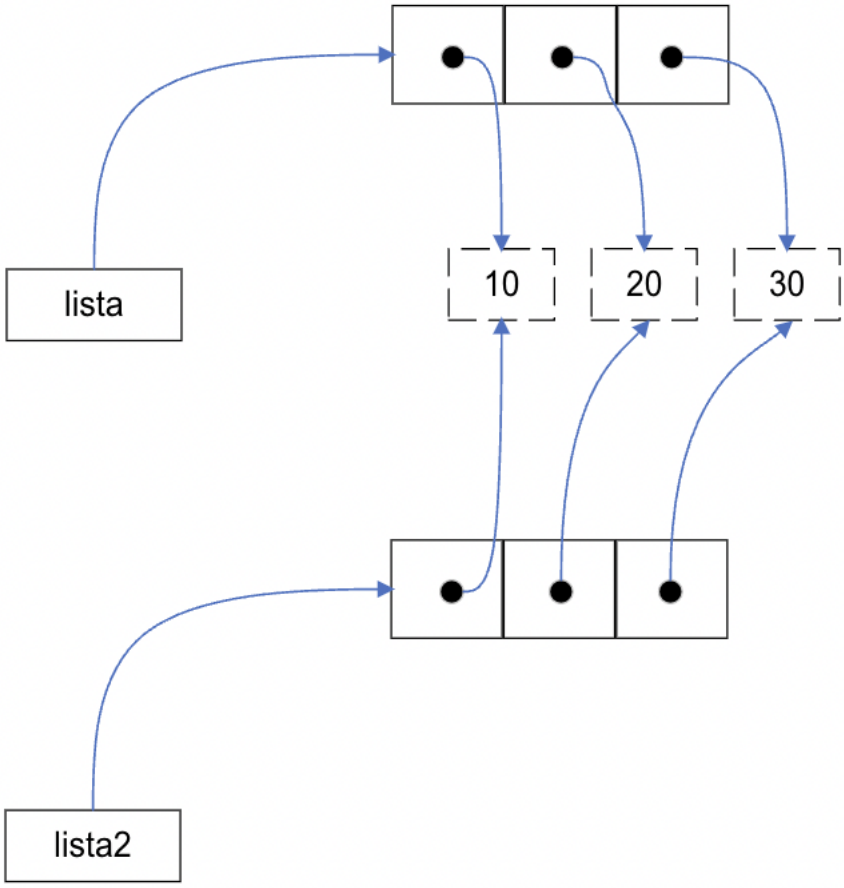
Una copia superficiale di una lista è una **nuova lista che contiene riferimenti agli stessi oggetti presenti nella lista originale**. Python crea un nuovo oggetto lista, ma inserisce al suo interno i riferimenti agli stessi elementi contenuti nella lista di partenza. Questo significa che, **anche se la nuova lista è indipendente come contenitore, gli oggetti al suo interno non sono duplicati**: sono condivisi tra la lista originale e la copia.

L'operazione

```
lista2 = lista[:]
```

crea una copia superficiale della lista. Esaminiamo alcuni esempi in dettaglio.

```
lista = ['paperino', 'pippo', 'topolino']
lista2 = lista[:]
print(lista == lista2) # True: il contenuto è uguale
print(id(lista[0]) == id(lista2[0])) # True: il primo elemento è lo stesso
print(id(lista) == id(lista2)) #False: le due liste sono oggetti diversi
```



Esistono almeno tre modi per creare una copia superficiale di una lista in Python:

```
# L'operatore di slicing [:]
lista2 = lista[:]
```

```
# Il metodo .copy() delle liste
lista2 = lista.copy()

# La funzione `copy()` del modulo copy

import copy
lista2 = copy.copy(lista)
```

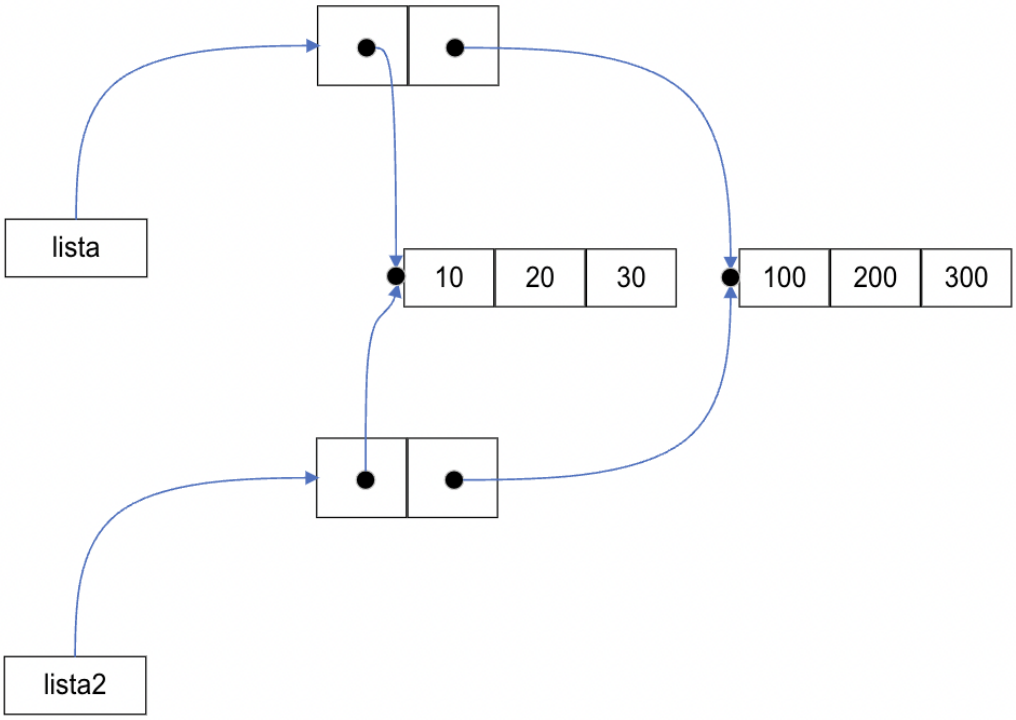
I tre modi sono sostanzialmente uguali per quanto rigrarda le liste. La funzione ed il metodo `copy` saranno utili quando faremo delle copie superficiali di oggetti diversi dalle liste, per i quali lo slicing non è applicabile.

Gli effetti della copia superficiale emergono nettamente quando gli elementi della lista sono a loro volta elementi *mutabili*, per esempio altre liste

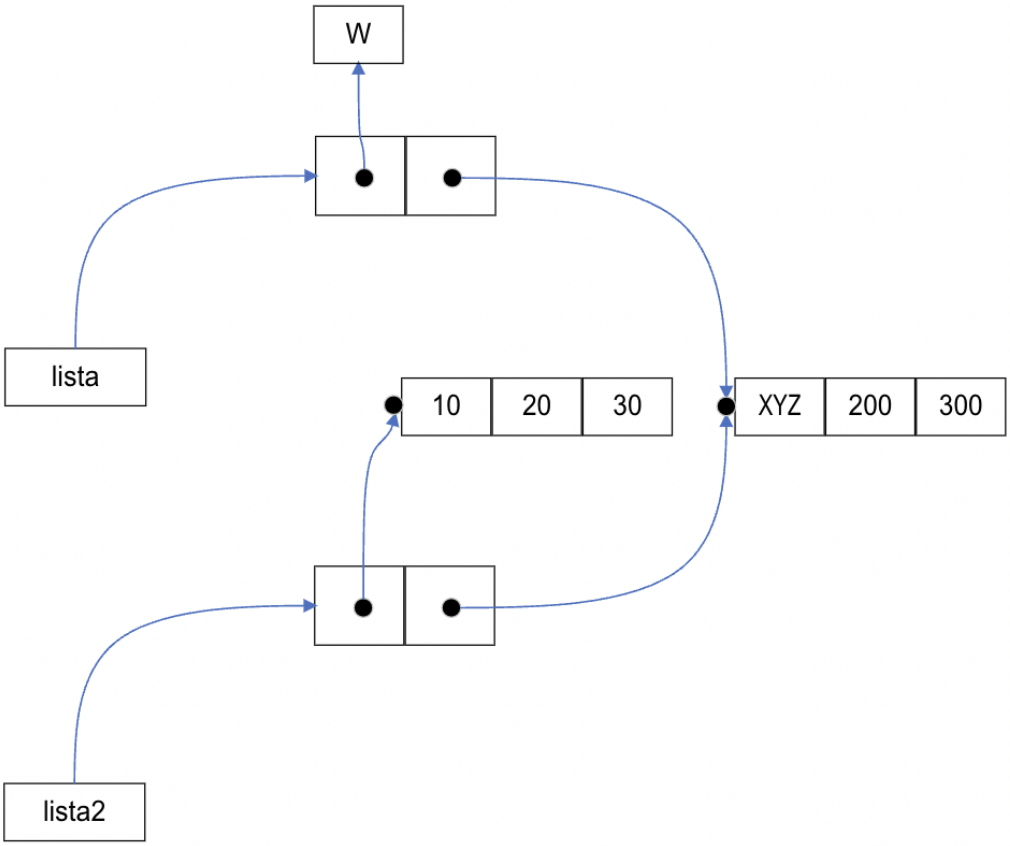
```
lista = [ [10, 20, 30], [100, 200, 300]]
lista2 = lista[:]
lista[0] = `W`
lista[1][0] = 'XYZ'
```

L'esempio mostra che le due liste **non** sono indipendenti: il primo elemento cambia solo per `lista`; il secondo elemento cambia per entrambe.

```
lista = [ [10, 20, 30], [100, 200, 300]]
lista2 = lista[:]
```



```
lista[0]='W'
lista[1][0] = 'XYZ'
```



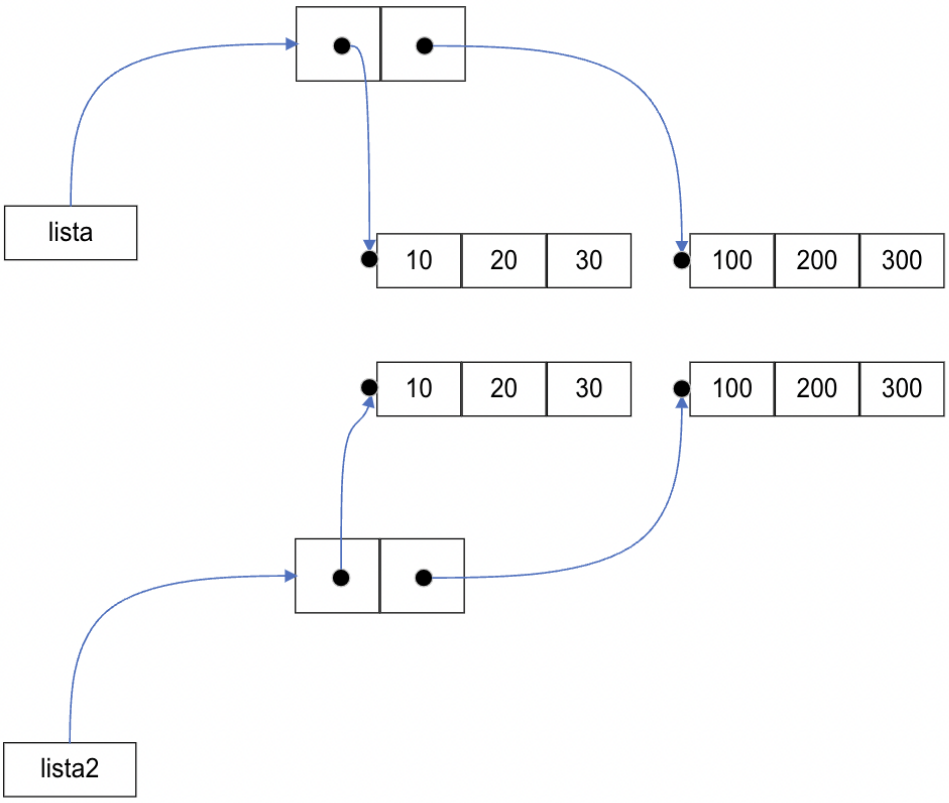
Deep Copy

Una **deep copy** è una copia completa e indipendente di una lista (in genere, di un oggetto), compresi tutti gli oggetti annidati al suo interno. Le modifiche fatte alla copia non influenzano l'originale (e viceversa).

```
import copy
lista = [ [10, 20, 30], [100, 200, 300]]
lista2 = copy.deepcopy(lista)

lista[0] = `W`
lista[1][0] = 'XYZ'
```

```
lista = [ [10, 20, 30], [100, 200, 300]]
lista2 = copy.deepcopy(lista)
```



https://docs.python.org/3/tutorial/datastructures.html#more-on-lists

Principali metodi e funzioni per le liste

Metodo / Funzione	Tipo	Descrizione	Esempio
`append(x)`	metodo	Aggiunge l'elemento `x` alla fine della lista	`L.append(4)`
`extend(iter)`	metodo	Estende la lista aggiungendo tutti gli elementi di `iter`	`L.extend([4,5])`
`remove(x)`	metodo	Rimuove la prima occorrenza di `x`; solleva `ValueError` se non trovato	`L.remove(2)`
`clear()`	metodo	Rimuove tutti gli elementi	`L.clear()`

Metodo / Funzione	Tipo	Descrizione	Esempio
`copy()`	metodo	Restituisce una copia superficiale (shallow copy)	`M = L.copy()`
`len(L)`	funzione	Restituisce il numero di elementi	`len(L)`
`sum(L)`	funzione	Somma tutti gli elementi numerici	`sum([1,2,3])`
`min(L)`	funzione	Restituisce il valore minimo	`min([1,2,3])`
`max(L)`	funzione	Restituisce il valore massimo	`max([1,2,3])`
`any(L)`	funzione	`True` se almeno un elemento è `True` (come in OR)	`any([0, '', 5]) # True`
`all(L)`	funzione	`True` se tutti gli elementi sono `True` (come in AND)	`all([1, 'a', True]) # True`

Considerazioni finali

Per lavorare correttamente con le liste è fondamentale padroneggiare alcune operazioni di base.
È necessario saper **iterare sugli elementi**, sia in avanti sia all'indietro, eventualmente con passo personalizzato, utilizzando costrutti come `for` e `while`. Occorre inoltre saper **aggiungere elementi** alla lista e costruirne di nuove a partire da quelle esistenti.

Un aspetto importante è la capacità di **selezionare o eliminare elementi** in base a determinate condizioni: ad esempio tutti i valori negativi oppure gli elementi in posizioni specifiche (come gli indici pari). È essenziale distinguere chiaramente tra **condizioni sui valori** (contenuto degli elementi) e **condizioni sugli indici** (posizione nella lista).

Quando si rimuovono elementi durante l'iterazione, bisogna prestare particolare attenzione, perché la modifica della lista può alterare gli indici e portare a errori o comportamenti inattesi; in questi casi è spesso preferibile adottare strategie alternative (consultate la sezione sulle strutture iterative).

Quando si accede contemporaneamente a un elemento e a quello **precedente o successivo**, è necessario gestire correttamente i **casi limite**: il primo elemento non ha un precedente e l'ultimo non ha un successivo.

Le liste sono **strutture mutabili**: il loro contenuto può essere modificato senza creare un nuovo oggetto. Come per tutti gli oggetti in Python, le variabili contengono **riferimenti** agli oggetti; quindi un'assegnazione come `b = a` non crea una copia, ma fa sì che entrambe le variabili si riferiscano alla stessa lista. Poiché le liste sono mutabili, una modifica effettuata

tramite una variabile è visibile anche dall'altra. Per ottenere una copia indipendente è necessario crearla esplicitamente.

Considerazioni finali

- Saper **iterare** su una lista (in avanti, all'indietro, con passo variabile) usando `for` o `while`.
- Saper **aggiungere elementi** e costruire nuove liste.
- Saper **selezionare o eliminare elementi** in base a condizioni:
  - sul **valore** (es. numeri negativi)
  - sull'**indice** (es. posizioni pari)
    → non confondere le due cose.
- Prestare attenzione quando si **modifica una lista durante l'iterazione**: si rischiano errori o risultati inattesi.
- Quando si accede a elementi consecutivi (**precedente/successivo**), gestire i **casi limite** (primo e ultimo elemento).
- Le liste sono **mutabili**: più variabili possono riferirsi allo stesso oggetto (come per tutti i tipi in Python), quindi le modifiche sono condivise; per copie indipendenti serve una copia esplicita.